

ChatGPT ▾

✦ Upgrade

↑ Share ...

integer factoring as bipartite matching

Integer factorization can indeed be formulated as a **bipartite matching problem**, although this is usually a theoretical reformulation rather than a practical factoring algorithm.

Basic Idea

Suppose we want to factor:

$$N = p \times q$$

where p and q are unknown integers.

Represent the binary digits of p and q :

$$p = \sum_i p_i 2^i, \quad q = \sum_j q_j 2^j$$

with $p_i, q_j \in \{0, 1\}$.

Multiplication gives:

$$N = \sum_{i,j} p_i q_j 2^{i+j}$$

Each term $p_i q_j$ is a binary product.

Bipartite Graph Construction

Create a bipartite graph:

- Left vertices: bits of p ($p_0, p_1, \dots$)
- Right vertices: bits of q ($q_0, q_1, \dots$)

An edge (i, j) represents the product term:

$$e_{ij} = p_i q_j$$

with weight

$$w_{ij} = 2^{i+j}$$

The factorization condition becomes:

$$\sum_{(i,j)} w_{ij} e_{ij} = N$$

subject to binary carry constraints.

The graph is therefore a weighted bipartite graph whose selected edges must reproduce the binary expansion of N .

Relation to Matching

If we define:

$$e_{ij} = 1 \iff p_i = q_j = 1$$

then the edge set corresponds to the Cartesian product of the 1-bits in p and q .

Let

$$P = \{i : p_i = 1\}, \quad Q = \{j : q_j = 1\}$$

The active edges form a complete bipartite subgraph:

$$K_{|P|,|Q|}$$

Factorization becomes finding subsets P, Q whose weighted edge contributions match N .

This can be viewed as:

- a constrained bipartite matching problem,
 - a biclique-finding problem,
 - a 0–1 integer programming problem.
-

Example: $21 = 3 \times 7$

Binary:

$$3 = 11_2$$

$$7 = 111_2$$

Left nodes:

$$p_0, p_1$$

Right nodes:

$$q_0, q_1, q_2$$

Selected edges:

$$(p_0, q_0), (p_0, q_1), (p_0, q_2),$$

$$(p_1, q_0), (p_1, q_1), (p_1, q_2)$$

with weights

$$1, 2, 4, 2, 4, 8.$$

Adding with carries yields

$$10101_2 = 21.$$

The active edges form a biclique $K_{2,3}$.

SAT and Matching Connection

Modern factorization-to-SAT reductions do something very similar:

1. Build a multiplication circuit.
2. Introduce variables for every partial product $p_i q_j$.
3. Add carry constraints.
4. Solve as SAT.

The resulting factor graph is naturally bipartite:

- factor bits on one side,
- partial products on the other.

Many SAT, QUBO, and Ising formulations used in quantum annealing derive from this structure.

Complexity

A pure maximum bipartite matching algorithm (e.g., Hopcroft–Karp) does **not** factor integers efficiently because:

- carries couple many edges together,
- constraints are nonlinear,
- the graph is not simply asking for a maximum cardinality matching.

Instead, factorization becomes a constrained biclique/Boolean optimization problem, which remains computationally difficult.

In complexity-theoretic terms:

Factoring $\rightarrow$ Integer Programming $\rightarrow$ Bipartite Constraint Graph

but no polynomial-time reduction to ordinary bipartite matching is known.

A More Algebraic View

The multiplication table can be written as a rank-1 binary matrix:

$$M_{ij} = p_i q_j$$

Factorization asks whether the weighted matrix M , after diagonal summation and carry propagation, produces N .

Thus integer factorization can also be viewed as recovering two binary vectors whose outer product satisfies a set of linear and carry constraints. This connects factorization to:

- bipartite graph reconstruction,
- biclique discovery,
- low-rank Boolean matrix factorization,

- SAT/QUBO formulations,
- tensor decomposition methods.

This viewpoint is useful in research on SAT-based factoring, quantum annealing formulations, and algebraic cryptanalysis, although it has not led to an algorithm that outperforms the best classical methods such as the General Number Field Sieve.

what are the implications of recent result "bipartite matching is in NC" for above reduction

The recent result that **bipartite matching is in NC** is a major breakthrough in parallel complexity, but its implications for integer factorization are more subtle than they might first appear.  [eccc.weizmann....](#)

Short answer

If your reduction from factorization to a bipartite graph produces an **ordinary bipartite matching instance**, then factorization would also be in NC.

However, the known factorization formulations do **not** reduce to plain matching. They reduce to matching **plus nonlinear carry constraints**, which are the hard part.

So the new result does not immediately imply that factoring is in NC.

Where the difficulty really lies

Recall the binary multiplication table:

$$N = p \times q$$

with partial products

$$m_{ij} = p_i q_j.$$

The bipartite graph naturally represents these terms:

- left side = bits of p
- right side = bits of q
- edge (i, j) = partial product m_{ij}

The obstacle is that integer multiplication is not simply:

$$N = \sum m_{ij} 2^{i+j}.$$

There are carries:

$$s_k + c_k = N_k + 2c_{k+1}$$

where

$$s_k = \sum_{i+j=k} m_{ij}.$$

These carry variables couple many edges simultaneously.

Ordinary matching algorithms are excellent at enforcing:

- one-to-one constraints,
- degree constraints,

- linear weight optimization.

Carries create higher-order constraints that are not matching constraints.

Complexity-Theoretic Interpretation

The new theorem says:

$$\text{Bipartite Matching} \in NC$$

deterministically.  [eccc.weizmann....](#)

For factoring to inherit this, one would need:

$$\text{Factoring} \leq_{NC} \text{Bipartite Matching}$$

with a reduction that preserves solutions.

No such reduction is known.

The standard reductions are closer to:

$$\text{Factoring} \leq \text{SAT}$$

or

$$\text{Factoring} \leq \text{Integer Programming}$$

than to matching.

A More Interesting Possibility

Your bipartite formulation suggests something deeper.

Factorization can be viewed as finding a rank-1 binary matrix:

$$M = pq^T$$

whose diagonal sums reproduce N .

The matching breakthrough also established NC algorithms for related algebraic objects such as weighted bipartite matching and noncommutative rank.  [eccc.weizmann....](#)

That raises a research question:

Can multiplication constraints be expressed through rank, determinant, or matroid structures that are now known to be in NC?

If yes, the new techniques might become relevant to factorization indirectly.

Biclique Perspective

A factorization corresponds to a biclique:

$$K_{|P|,|Q|}$$

where

$$P = \{i : p_i = 1\}, \quad Q = \{j : q_j = 1\}.$$

The active partial products are exactly the edges of this biclique.

Matching finds sparse structures:

$$O(n)$$

edges.

Factorization seeks a structured dense subgraph:

$$|P||Q|$$

edges plus carry consistency.

This is much closer to:

- biclique finding,
- Boolean matrix factorization,
- tensor decomposition,

than to matching itself.

The recent NC result says little directly about those problems.

What Would Be Revolutionary

Suppose someone showed:

$$\text{Factoring} \leq_{NC} \text{Weighted Bipartite Matching}$$

or

$$\text{Factoring} \leq_{NC} \text{Matroid Intersection.}$$

Then, combining that with the new theorem would place factoring in NC immediately.  [eccc.weizmann....](#)

That would imply highly parallel deterministic factoring algorithms, which would be a major complexity-theoretic advance.

At present, there is no known reduction of this strength.

My assessment

The new "Bipartite Matching is in NC" result is probably **more interesting for factorization research than for practical factoring** because it removes one long-standing obstacle:

- Matching itself is no longer the bottleneck.
- Any future factorization reduction that lands entirely inside matching/matroid/rank computations would automatically inherit efficient parallel algorithms.

The remaining challenge is the carry structure. If someone discovers a way to encode binary carries as a matching-compatible algebraic invariant, then the new NC techniques could become unexpectedly relevant to integer factorization. Right now, that step remains open.  [eccc.weizmann....](#)